



**INSTITUTE OF MATHEMATICS AND
COMPUTER SCIENCE
UNIVERSITY OF SINDH , JAMSHORO**



**NAME : MALIHA
DEPARTMENT : COMPUTER SCIENCE
SUBJECT : HCI AND COMPUTER GRAPHICS
PART III ROLL NO : 2K24/CSE/82
ASSIGNED BY : SIR RAJESH KUMAR**

ASSIGNMENT NO : 2

Task 1: Display Pixel Density (PPI/DPI) Calculator

```
import math

# User input
Wpx = int(input("Enter horizontal resolution (pixels): "))
Hpx = int(input("Enter vertical resolution (pixels): "))
Dinches = float(input("Enter physical diagonal size (inches): "))

# Total pixels
total_pixels = Wpx * Hpx

# Simplified aspect ratio
gcd = math.gcd(Wpx, Hpx)
aspect_w = Wpx // gcd
aspect_h = Hpx // gcd

# Calculate PPI/DPI
diagonal_pixels = math.sqrt(Wpx**2 + Hpx**2)
dpi = diagonal_pixels / Dinches

# Density classification
```

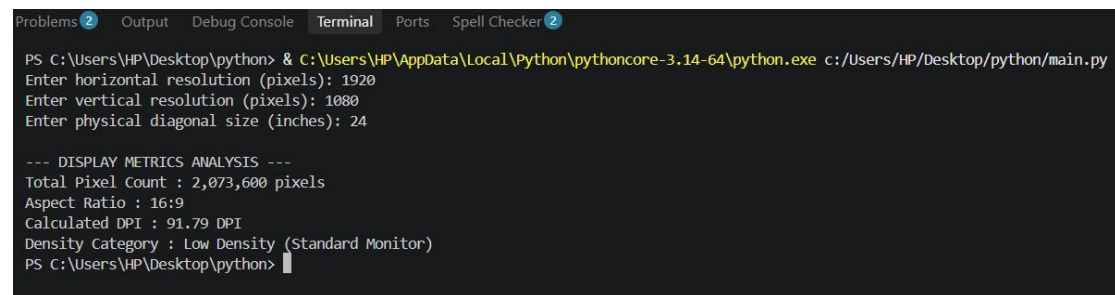
```

if dpi < 100:
    density = "Low Density (Standard Monitor)"
elif dpi <= 200:
    density = "Medium Density (HD Display)"
else:
    density = "High Density (Retina / Mobile)"

# Output
print("\n--- DISPLAY METRICS ANALYSIS ---")
print(f"Total Pixel Count : {total_pixels:,} pixels")
print(f"Aspect Ratio : {aspect_w}:{aspect_h}")
print(f"Calculated DPI : {dpi:.2f} DPI")
print(f"Density Category : {density}")

```

output



```

Problems 2 Output Debug Console Terminal Ports Spell Checker 2
PS C:\Users\HP\Desktop\python> & C:\Users\HP\AppData\Local\Python\pythoncore-3.14-64\python.exe c:/Users/HP/Desktop/python/main.py
Enter horizontal resolution (pixels): 1920
Enter vertical resolution (pixels): 1080
Enter physical diagonal size (inches): 24

--- DISPLAY METRICS ANALYSIS ---
Total Pixel Count : 2,073,600 pixels
Aspect Ratio : 16:9
Calculated DPI : 91.79 DPI
Density Category : Low Density (Standard Monitor)
PS C:\Users\HP\Desktop\python>

```

Task 2: Environment Setup & Synthetic Image Matrix Creation

```

import numpy as np
import matplotlib.pyplot as plt

# Create 300 x 400 RGB image
img = np.zeros((300, 400, 3), dtype=np.uint8)

# Top-left = Red
img[:150, :200] = [255, 0, 0]

# Top-right = Green
img[:150, 200:] = [0, 255, 0]

# Bottom-left = Blue
img[150:, :200] = [0, 0, 255]

# Bottom-right = White
img[150:, 200:] = [255, 255, 255]

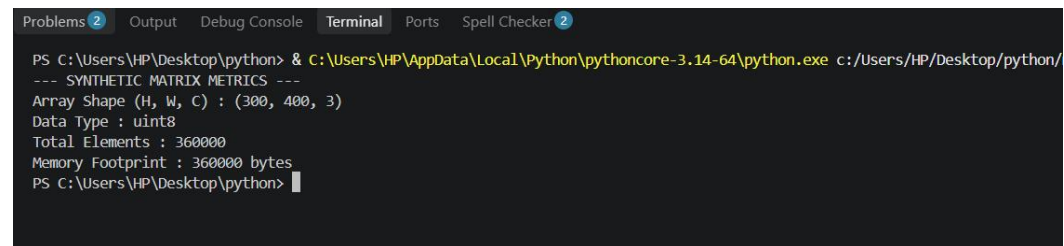
# Print information
print("--- SYNTHETIC MATRIX METRICS ---")
print("Array Shape (H, W, C) :", img.shape)
print("Data Type :", img.dtype)
print("Total Elements :", img.size)

```

```
print("Memory Footprint :", img.nbytes, "bytes")
```

```
# Display image
plt.imshow(img)
plt.title("Synthetic 300 x 400 RGB Image")
plt.axis("off")
plt.show()
```

output



```
Problems 2 Output Debug Console Terminal Ports Spell Checker 2
PS C:\Users\HP\Desktop\python> & C:\Users\HP\AppData\Local\Python\pythoncore-3.14-64\python.exe c:/Users/HP/Desktop/python/
--- SYNTHETIC MATRIX METRICS ---
Array Shape (H, W, C) : (300, 400, 3)
Data Type : uint8
Total Elements : 360000
Memory Footprint : 360000 bytes
PS C:\Users\HP\Desktop\python> |
```

Task 3: Channel Slicing & Isolation

```
import numpy as np
import matplotlib.pyplot as plt
from PIL import Image

# Load image
image = Image.open("sample.jpg").convert("RGB")

# Convert image to NumPy array
img = np.array(image)

# Extract individual channels
red = img[:, :, 0]
green = img[:, :, 1]
blue = img[:, :, 2]

# Create red-only image
red_only = np.zeros_like(img)
red_only[:, :, 0] = red

# Create green-only image
green_only = np.zeros_like(img)
green_only[:, :, 1] = green

# Create blue-only image
blue_only = np.zeros_like(img)
blue_only[:, :, 2] = blue

# Print summary
print("--- CHANNEL EXTRACTION SUMMARY ---")
```

```

print("Original Image Shape :", img.shape)

print("Red Channel 2D Shape :", red.shape,
      "| Mean Intensity:", red.mean())

print("Green Channel 2D Shape :", green.shape,
      "| Mean Intensity:", green.mean())

print("Blue Channel 2D Shape :", blue.shape,
      "| Mean Intensity:", blue.mean())

# Display 2 x 3 layout
plt.figure(figsize=(12, 7))

plt.subplot(2, 3, 1)
plt.imshow(red_only)
plt.title("Red Only")
plt.axis("off")

plt.subplot(2, 3, 2)
plt.imshow(green_only)
plt.title("Green Only")
plt.axis("off")

plt.subplot(2, 3, 3)
plt.imshow(blue_only)
plt.title("Blue Only")
plt.axis("off")

plt.subplot(2, 3, 4)
plt.imshow(red, cmap="gray")
plt.title("Red Intensity")
plt.axis("off")

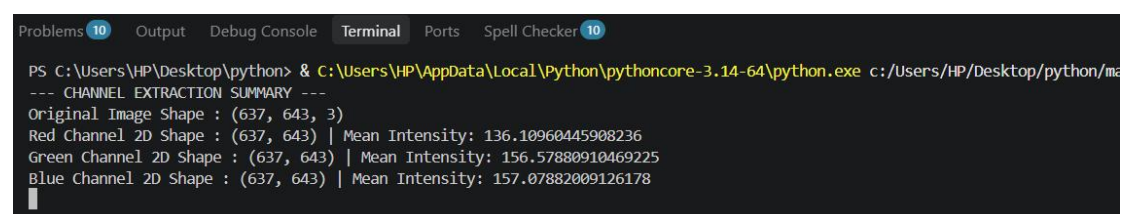
plt.subplot(2, 3, 5)
plt.imshow(green, cmap="gray")
plt.title("Green Intensity")
plt.axis("off")

plt.subplot(2, 3, 6)
plt.imshow(blue, cmap="gray")
plt.title("Blue Intensity")
plt.axis("off")

plt.tight_layout()
plt.show()

```

output



```

Problems 10 Output Debug Console Terminal Ports Spell Checker 10
PS C:\Users\HP\Desktop\python> & C:\Users\HP\AppData\Local\Python\pythoncore-3.14-64\python.exe c:/Users/HP/Desktop/python/ma
--- CHANNEL EXTRACTION SUMMARY ---
Original Image Shape : (637, 643, 3)
Red Channel 2D Shape : (637, 643) | Mean Intensity: 136.10960445908236
Green Channel 2D Shape : (637, 643) | Mean Intensity: 156.57880910469225
Blue Channel 2D Shape : (637, 643) | Mean Intensity: 157.07882009126178

```

Task 4: Spatial Downsampling & Pixelation via Striding

```
import numpy as np
import matplotlib.pyplot as plt
from PIL import Image

# Load image
image = Image.open("sample.jpg").convert("RGB")
img = np.array(image)

N = 8

# Downsample
downsampled = img[::N, ::N, :]

# Re-expand using repeat
expanded = np.repeat(
    np.repeat(downsampled, N, axis=0),
    N,
    axis=1
)

# Crop to original dimensions
expanded = expanded[:img.shape[0], :img.shape[1], :]

# Memory reduction
memory_reduction = (
    1 - downsampled.nbytes / img.nbytes
) * 100

# Dimension reduction
height_reduction = (
    1 - downsampled.shape[0] / img.shape[0]
) * 100

width_reduction = (
    1 - downsampled.shape[1] / img.shape[1]
) * 100

print("--- DOWNSAMPLING ANALYSIS (N = 8) ---")
print("Original Shape :", img.shape)
print("Memory :", img.nbytes, "bytes")

print("Downsampled Shape :", downsampled.shape)
print("Memory :", downsampled.nbytes, "bytes")

print("Re-expanded Shape :", expanded.shape)

print(f"Height Reduction : {height_reduction:.2f}%")
print(f"Width Reduction : {width_reduction:.2f}%")
print(f"Memory Savings : {memory_reduction:.2f}%")
```

```

# Display
plt.figure(figsize=(12, 5))

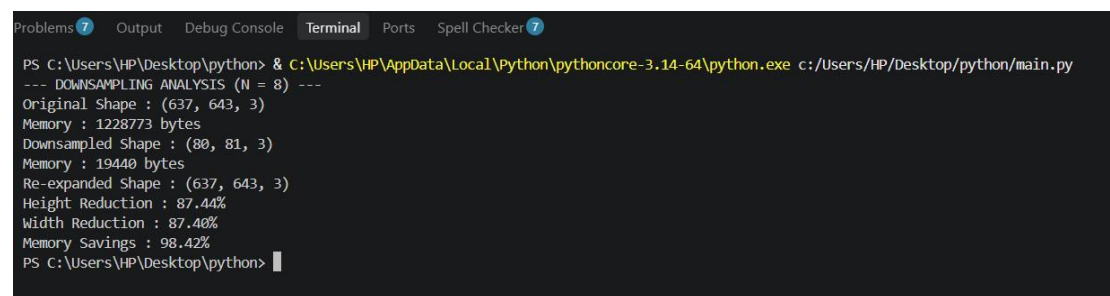
plt.subplot(1, 2, 1)
plt.imshow(img)
plt.title("Original Image")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(expanded)
plt.title("Pixelated Image")
plt.axis("off")

plt.tight_layout()
plt.show()

```

output



```

Problems 7 Output Debug Console Terminal Ports Spell Checker 7
PS C:\Users\HP\Desktop\python> & C:\Users\HP\AppData\Local\Python\pythoncore-3.14-64\python.exe c:/Users/HP/Desktop/python/main.py
--- DOWNSAMPLING ANALYSIS (N = 8) ---
Original Shape : (637, 643, 3)
Memory : 1228773 bytes
Downsampled Shape : (80, 81, 3)
Memory : 19440 bytes
Re-expanded Shape : (637, 643, 3)
Height Reduction : 87.44%
Width Reduction : 87.40%
Memory Savings : 98.42%
PS C:\Users\HP\Desktop\python>

```

□ **Short Answer: Why does adding an Alpha channel (RGBA) increase an image array's memory consumption by 33% compared to standard RGB?**

Adding an Alpha channel increases the memory consumption because a standard RGB image stores 3 values per pixel: Red, Green, and Blue. When we use RGBA, an extra Alpha value is added to store the pixel's transparency, making it 4 values per pixel. Therefore, memory increases from 3 units to 4 units, which is about 33.33% more memory than RGB.